

Robotics Multi-Agent Assistant

A Production Multi-Agent System for Robotics Hardware, Wiring, Code, Purchasing, and 3D Visualization

Final Project Report — Production AI Agent System

Table of Contents

1. [Executive Summary](#)
2. [System Overview](#)
3. [Architecture](#)
 - 3.1 [Service Topology](#)
 - 3.2 [Why the System Is Split This Way](#)
 - 3.3 [API Layer](#)
 - 3.4 [Session Management](#)
4. [The Agent Roster](#)
 - 4.1 [The Supervisor](#)
 - 4.2 [RAG Agent](#)
 - 4.3 [Wiring Agent](#)
 - 4.4 [Coding Agent](#)
 - 4.5 [Robot Visualization Agent](#)
 - 4.6 [Component Manager \(System B\)](#)
5. [The RAG Pipeline](#)
 - 5.1 [Ingestion and Chunking](#)
 - 5.2 [Embedding and Vector Storage](#)
 - 5.3 [Retrieval: Hybrid Search and Reranking](#)
 - 5.4 [Multimodal Understanding: Image Captioning](#)
 - 5.5 [Grounded Generation](#)
 - 5.6 [End-to-End Pipeline Diagram](#)
6. [Trust and Safety](#)
 - 6.1 [Guardrails](#)
 - 6.2 [Human-in-the-Loop Approval](#)
 - 6.3 [Payment Security](#)
7. [Evaluation Methodology](#)

8. RAG Evaluation

- 8.1 Retrieval Ranking Metrics
- 8.2 Top-K Comparison
- 8.3 Generation Quality (RAGAS)
- 8.4 Configuration Comparisons

9. Agent Evaluation

- 9.1 Routing and Tool Selection
- 9.2 Correctness, LLM-as-Judge, and Pricing
- 9.3 Held-Out Validation — Is the 100% Real?

10. Failure Case Analysis

11. Known Limitations

12. Conclusion and Future Work

1. Executive Summary

This project is a production-oriented multi-agent AI system that helps a user go from a robotics idea to a working, sourced, documented build. A user can ask it to explain how a sensor works, plan and validate a wiring layout for a microcontroller board, generate and validate Arduino/embedded code, render a 3D preview of a robot design, and search for and purchase electronic components — all through one conversational interface, with every side-effecting action gated behind an explicit human approval step.

The system demonstrates the full stack required of a production agent system: two independently deployed agent frameworks (LangGraph and Google ADK) communicating only over the network; a retrieval-augmented generation (RAG) pipeline grounded in real Arduino/robotics documentation; a custom MCP server exposing that RAG pipeline as tool calls; input/output guardrails and human-in-the-loop approval compiled directly into the agent graph; a FastAPI layer with both request/response and Server-Sent-Events (SSE) streaming endpoints; and a rigorous, two-track evaluation methodology covering retrieval quality, generation quality, agent routing and tool-selection accuracy, and a deliberate held-out stress test built specifically to check whether the system's near-perfect scores were genuine or an artifact of having been tuned against its own test set.

Five independently startable Docker containers make up the deployed system: the LangGraph-based primary agent (`agent-system-a`), the Google ADK-based Component Manager (`agent-system-b`), a custom MCP server exposing the RAG pipeline, a Qdrant vector database, and a React/TypeScript frontend — all orchestrated by one `docker-compose up` command.

2. System Overview

What it does. A user chats with the assistant about a robotics build. Depending on what's asked, the assistant's supervisor routes the request to one of five specialists: documentation lookup, pin/wiring planning, code generation, 3D model rendering, or component sourcing and purchasing. Any specialist that would write a file, run a render, or spend money first pauses the conversation and asks for explicit approval before proceeding.

Who it's for. Hobbyists and students building Arduino/Raspberry Pi-class robotics projects who want one assistant that can explain a part, plan how to wire it, write and validate the code that drives it, show what the finished build looks like, and — if it isn't already on hand — find and buy it, without switching between a datasheet, a wiring diagram tool, an IDE, a CAD tool, and a distributor's website.

What makes it a "production" system, not a demo. Three things distinguish it from a single-prompt chatbot wrapper:

- **A real network boundary.** The purchasing specialist is not a Python function inside the main application — it is a completely separate service (a different framework, a different language runtime philosophy, a different team's ownership model in principle), reached only through the Agent-to-Agent (A2A) protocol or REST, the same way a real company's agent would call a partner company's agent.
- **Nothing happens without a human.** Every side-effecting action — writing code to disk, rendering a 3D model, spending sandbox money — pauses the LangGraph execution at an `interrupt()` node and waits for an explicit approve/reject decision before continuing.
- **The system is evaluated, not just demonstrated.** Every claim made in this report about accuracy, cost, or quality is backed by a saved, reproducible evaluation run under `evaluation/runs/`, not an anecdote.

3. Architecture

3.1 Service Topology

Container	Framework	Role
agent-system-a	LangGraph + FastAPI (Python)	Primary system: supervisor, specialists, guardrails, approvals, payments, public API
agent-system-b	Google ADK (Python)	Component Manager — DigiKey sourcing and sandbox ordering, reached only over the network
mcp-server	FastMCP (Python)	Exposes the robotics RAG pipeline as MCP tools over Streamable HTTP
vector-db	Qdrant	Vector storage for the RAG corpus
frontend	React + TypeScript + Vite, served by nginx	The chat UI a user actually talks to

Each container has its own `Dockerfile` and its own dependency file, and can be built and started independently. The whole stack starts with one command: `docker compose up -d --build`.

3.2 Why the System Is Split This Way

Production AI systems do not exist in isolation — a real travel-booking agent calls a hotel-booking agent owned by a different company, written in a different stack, deployed on a different release cycle. This project takes that seriously rather than simulating it: **agent-system-a never imports agent-system-b's Python package**. The Component Manager is reached in two ways, both over the network:

- The core purchasing flow (search, price, propose, order) goes over the **A2A protocol** (JSON-RPC, port 8002) — System A sends a plain-text task, System B's own ADK agent independently decides which of its own tools to call and in what order, and System A only ever reads the result. System A never second-guesses which purchasing step to take; that decision belongs entirely to System B.
- The DigiKey OAuth login flow proxies over System B's own **REST API** (port 8003) instead.

This means the two systems could be deployed by two different teams, in two different languages, on two different schedules, and still work — which is exactly the property a single in-process "purchasing module" would not have.

Ownership breaks down cleanly along these lines:

- **Agent System A** owns conversation state, routing, human approvals, and payment tokenization.
- **Agent System B** owns DigiKey OAuth and sandbox ordering.
- **The MCP service** owns the embedded robotics RAG runtime and its corpus.
- **The browser only ever talks to Agent System A's public API** — it never reaches System B, the MCP server, or Qdrant directly.

3.3 API Layer

`agent-system-a` exposes a FastAPI application with, among others:

Endpoint	Purpose
<code>POST /threads</code>	Create a new conversation thread
<code>GET /threads</code>	List recent conversations
<code>GET /threads/{id}/messages</code>	Full public history for a thread
<code>POST /threads/{id}/messages</code>	Send a message (blocks until the turn finishes or pauses)
<code>POST /threads/{id}/messages/stream</code>	The same, but streamed over Server-Sent Events as each LangGraph node completes
<code>POST /threads/{id}/resume / .../resume/stream</code>	Resolve a pending approval, unary or streamed
<code>POST /payments/sandbox/tokenize</code>	Tokenize a sandbox test card without the raw PAN ever entering LangGraph state
<code>GET /auth/digikey/start</code> , <code>GET /auth/digikey/callback</code>	DigiKey sandbox OAuth, proxied through System B
<code>GET /artifacts/{id}</code>	Serve a generated image without exposing filesystem paths

The streaming endpoints were added specifically so the Activity panel in the UI reflects what the agent is doing *as it happens* — which node is running, which tools it called — instead of a static "thinking..." placeholder followed by one blob of output at the very end. Every specialist node's update is pushed to the client the moment it completes; the final answer itself is still only released after the output guardrail has checked it, so live progress never means unguarded content reaching the user early.

3.4 Session Management

Every conversation is a LangGraph checkpointed thread, persisted through an `AsyncSqliteSaver` keyed by `thread_id`. This means:

- Conversation history, the last wiring plan, any pending purchase, and any paused approval survive a page reload or an API restart.
- A paused `interrupt()` is not an in-memory flag — it is a real checkpointed graph state that can be resumed from a completely different process, minutes or hours later, with `Command(resume={...})`.

4. The Agent Roster

Each specialist has a narrow job, its own system prompt, and only the tools it needs — no specialist can reach outside its own toolset.

4.1 The Supervisor

The supervisor is not a specialist itself — it is the routing brain of the LangGraph. On every turn it looks at the original request, the recent conversation, which routes have already been tried, and what each specialist has produced so far, and picks exactly one next step: one of the five specialists, or `FINISH`. It never does specialist work itself.

Its prompt encodes several non-obvious rules learned from real failures during this project (see §10): route to `wiring_agent` *before* `coding_agent` or the visualization agent whenever a request names components to wire/code/model, so pin assignments exist before code or a model reuses them; never bundle multiple distinct purchase items into one purchasing task, since the purchasing flow proposes and pauses for approval one item at a time by design; and never re-invoke a specialist just to redisplay an artifact (a rendered image, a block of code) that was already produced and delivered earlier in the same thread.

4.2 RAG Agent

Model: Gemini 3.1 Flash Lite. **Tools:** `answer_question`, `show_image` — both supplied dynamically by the MCP server (see §5), not hard-coded into the agent itself.

Handles factual questions about Arduino and physical robotics hardware — a sensor's behavior, wiring conventions, motor control, documented electrical specs. Its prompt requires it to use the MCP tools for every documentation question, never invent information missing from retrieved documents, and proactively call `show_image` when a retrieved context has a relevant diagram — even if the user didn't explicitly ask to see it.

4.3 Wiring Agent

Model: Qwen3.6-27B. **Tools:** `get_board`, `get_component`, `allocate_pins`, `validate_wiring`, `format_wiring_plan`.

Plans and validates pin assignments for a named board (Arduino Uno, Arduino Mega 2560, ESP32, or Raspberry Pi 5) and one or more components from a 51-component catalog spanning sensors, motors, displays, and communication modules. Its prompt requires the tools to be called in a strict order — board lookup, then component lookup, then deterministic pin allocation, then validation, then formatting — so that pin numbers are never invented by the model and every conflict or voltage-mismatch warning the allocator finds is surfaced, never silently dropped. Existing pin assignments for a thread are protected internally by the allocator itself, so asking to add one more component never silently reassigns pins already committed to code the user may have already written.

4.4 Coding Agent

Model: Qwen3.6-27B. **Tools:** `save_code`, `validate_code`.

Writes, fixes, reviews, or explains Arduino/embedded C++ and any script that reads or controls hardware this assistant discusses. A generic code request with no hardware connection (e.g. "sort this list in Python") is explicitly out of scope and routed to `FINISH` instead. Every code-writing call is gated behind

human approval before this agent ever runs (see §6.2) — the model has no tools other than saving and validating a file, and its final answer is required to include the complete code inline, not just a saved path, since — unlike a rendered image — code has no delivery channel to the user independent of the answer text itself (a real gap found and fixed during this project's evaluation cycle, see §10).

4.5 Robot Visualization Agent

Model: Gemini 3.1 Flash Lite. **Tools:** `save_model`, `render_model`.

Builds a simple parametric 3D robot model — a list of boxes, cylinders, and spheres with positions and rotations in millimeters — and renders a preview image, using `build123d` /OpenCascade rather than any external CAD binary. Like the coding agent, model creation is always gated behind human approval. The rendered preview reaches the user through the API's `image_urls` mechanism, independent of what the model's own text answer says — one of the two different delivery-channel designs contrasted in the failure analysis in §10.

4.6 Component Manager (System B)

Framework: Google ADK, a genuinely separate service (see §3.2). **Tools:** `search_digikey`, `create_digikey_proposal`, `get_digikey_order`.

Searches DigiKey's sandbox catalog, compares offers itself using only tool data (preferring an exact technical match in stock, using price as the tie-breaker), and — only when the user has explicitly asked to buy something by name and quantity — creates exactly one purchase proposal and then stops, by design, so System A's graph can pause for human approval before anything is spent. It never accepts or asks for a card number, CVV, or any other raw payment detail; those stay entirely on System A's side, isolated in a dedicated payment vault (see §6.3).

5. The RAG Pipeline

The RAG pipeline is the system's grounding in real Arduino/robotics documentation, exposed to the `rag_agent` specialist as MCP tools rather than being embedded directly into System A's process.

5.1 Ingestion and Chunking

Source documents are parsed using native PDF text extraction where available, falling back to OCR only where extraction is insufficient. Rather than splitting by page — which cuts a diagram away from the paragraph that explains it, and cuts a procedure in half at an arbitrary page boundary — chunking follows the document's own **heading hierarchy**: PDF bookmarks and detected headings identify chapter, project, and subsection boundaries. Small consecutive subsections are packed together only within the same project; oversized sections are split with overlap. Every image chunk retains a link to its parent text chunk, preserving the semantic relationship between a wiring diagram and the paragraph that explains it.

5.2 Embedding and Vector Storage

Each chunk is embedded with **Gemini Embedding 2** and stored in **Qdrant**, with a modality metadata field distinguishing text chunks from image-caption chunks so a query can filter by modality at retrieval time.

5.3 Retrieval: Hybrid Search and Reranking

Retrieval combines dense vector search with **BM25** and **Reciprocal Rank Fusion (RRF)** so that exact technical terms (a part number, a pin name) are weighted alongside semantic similarity, then re-scores the fused candidate set with a **local cross-encoder reranker**. The measured effect of each stage is quantified in §8.

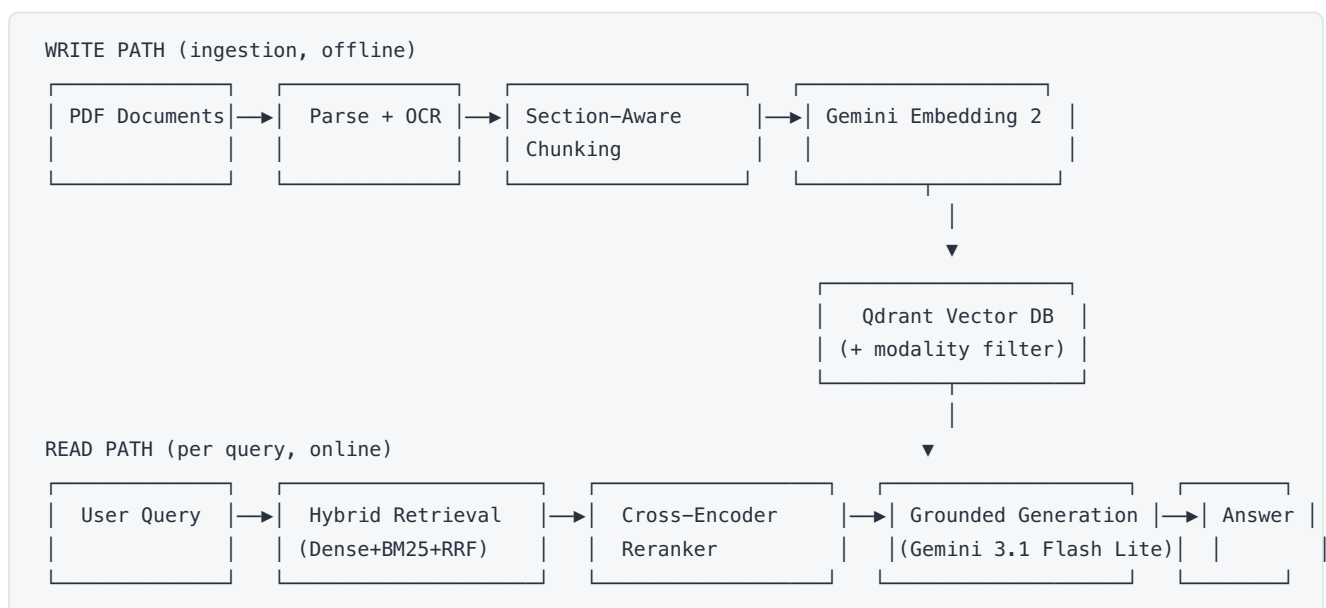
5.4 Multimodal Understanding: Image Captioning

Several early failure cases depended on a wiring diagram's pin mapping that text-only retrieval could never recover, even when it correctly identified the right project. Images are converted into **retrieval-focused technical captions** — text descriptions specific enough to be found by the same retrieval pipeline as any other chunk — while the original image file remains linked for direct display via the `show_image` tool. This was the single highest-leverage change measured in this project's RAG evaluation (see §8.4).

5.5 Grounded Generation

The final answer is produced by a prompt that instructs the model to answer **only** from the retrieved context, and to say so explicitly when the answer isn't there — the property that makes the faithfulness metric in §8.3 meaningful rather than cosmetic.

5.6 End-to-End Pipeline Diagram



6. Trust and Safety

6.1 Guardrails

Guardrails are compiled directly into the LangGraph as real nodes (`input_guardrail` , `output_guardrail`) — not bolted on as external middleware — using NVIDIA NeMo Guardrails. Two rails run on both input and output:

- **Regex-based sensitive-data masking**, run *before* anything reaches an LLM: email addresses, phone numbers, credit card numbers, US Social Security numbers, and IP addresses are masked with a `mask_sensitive_data` action before the input guardrail's self-check LLM ever sees the raw text.
- **A self-check LLM rail** (the same `guardrail_model()` instance used elsewhere, `openai/gpt-oss-safeguard-20b`) that can outright block a request, catching things a regex cannot, such as prompt-injection attempts ("ignore all previous instructions...").

A blocked input never reaches the supervisor at all — the graph's own conditional edge routes straight to `END` with a fixed refusal message. A blocked output is replaced with a fixed message rather than ever leaving the guardrail with unsafe content, and every finalized answer — including the supervisor's own conversational replies — passes through the output rail, not just specialist results.

6.2 Human-in-the-Loop Approval

Every side-effecting specialist — the coding agent, the visualization agent, and any DigiKey order submission — is gated by a dedicated graph node that calls LangGraph's `interrupt()` before the specialist runs. This is not a confirmation dialog bolted onto the UI: it is a real pause in graph execution, checkpointed to SQLite, resumable from any process with `Command(resume={"approved": ...})` . Rejecting cancels cleanly with no side effect — no file written, no render started, no order placed.

6.3 Payment Security

Every purchase requires an explicit, user-approved **AP2 payment mandate** validating the user, merchant, amount, items, expiry, nonce, and single-use constraint before the payment service ever runs. The LLM itself never sees a raw card number, CVV, or provider API key at any point — it only ever sees an opaque `payment_method_id` and a display string (e.g. "Visa ending in 4242"). Raw payment details are isolated in a dedicated payment vault, entirely outside LangGraph state, logs, or model context.

7. Evaluation Methodology

Two golden datasets are graded completely separately and never blended into one score:

- A **28-case agent dataset** (`evaluation/golden_dataset/v2/cases.jsonl`) covering routing, tool selection and ordering, guardrails, and approvals.
- A **50-question Arduino RAG dataset** covering retrieval ranking and RAGAS generation quality.

Within the agent dataset, every dimension is checked independently — route correctness, tool selection, tool call order, guardrail behavior, approval behavior, and project state are all separate pass/fail checks; a case's overall pass is their conjunction, never a single fuzzy proxy score. For cases with a stated goal, an **LLM judge** additionally grades whether the final answer actually accomplished it — deterministic checks can confirm the right tools ran, but only a judge can catch an answer that used the right tools and still failed to deliver (see §9.2). Every graph run is wrapped with a token-usage collector, priced against live provider rates, so cost and efficiency are measured, not estimated.

8. RAG Evaluation

8.1 Retrieval Ranking Metrics

Computed at K=5 over 100 questions, comparing the retriever before and after the local cross-encoder reranker:

Metric	Pre-reranker	Post-reranker	Change
Precision@5	0.660	0.696	+0.036
Recall@5	0.903	0.903	+0.000
MRR	0.852	0.877	+0.026
NDCG@5	0.734	0.760	+0.026

Reranking improved precision and ranking quality at **zero measured recall cost** — it reorders the same candidate set rather than discarding evidence.

8.2 Top-K Comparison

K	Precision@K	Recall@K	MRR	NDCG@K
3	0.767	0.940	0.892	0.807
5	0.720	0.960	0.892	0.788
8	0.670	0.960	0.892	0.779

K=3 wins on precision and NDCG — the tightest, cleanest context window. K=5 and K=8 tie on recall; pulling more chunks past K=5 stops finding new relevant evidence. MRR is flat at 0.892 across every K, confirming that K only changes how much extra context rides alongside the first relevant chunk, not that chunk's own rank. **K=5 is the system's chosen setting** — within 0.006 of K=8's recall, without diluting precision with an extra chunk of noise the way K=8 does.

8.3 Generation Quality (RAGAS)

Final architecture, full 30-question set:

Metric	Final mean	Coverage
Faithfulness	0.967	30/30
Answer relevancy	0.910	30/30
Context precision	0.779	30/30
Context recall	0.922	30/30

28 of 30 answers scored perfect faithfulness. Context precision remains the weakest dimension — a deliberate recall-favoring design choice (the retriever consistently finds the right section but sometimes keeps a neighboring one alongside it), not an unexamined gap.

8.4 Configuration Comparisons

Dense-only baseline vs. hybrid retrieval (RAGAS, 15 questions):

Metric	Dense baseline	Hybrid (BM25+RRF+reranker)	Change
Faithfulness	0.904	0.859	−0.045
Answer relevancy	0.878	0.805	−0.073
Context precision	0.605	0.702	+0.097
Context recall	0.967	0.833	−0.134

Hybrid retrieval hit its goal — context precision +0.097 — but introduced a real recall regression: query expansion grew the candidate pool, and the top-5 cutoff sometimes dropped a section a compound question still needed.

Baseline vs. detailed-caption pilot, same 15 questions:

Metric	Baseline	Detailed-caption pilot	Change
Faithfulness	0.904	0.944	+0.040
Answer relevancy	0.878	0.875	−0.003
Context precision	0.605	0.790	+0.185
Context recall	0.967	0.967	+0.000

Image captioning recovered the recall the hybrid-retrieval step had cost, while *also* improving precision further — every metric moved favorably with no tradeoff, which is why it became part of the final architecture rather than a rejected experiment.

Initial vs. final architecture (dense-only baseline vs. the complete hybrid+reranker+captions pipeline):

Metric	Initial (dense-only)	Final architecture	Δ
Faithfulness	0.904	0.967	+0.063
Answer relevancy	0.878	0.910	+0.032
Context precision	0.605	0.779	+0.174
Context recall	0.967	0.922	−0.045*

* Not a regression: the final number is scored on a broader 30-question set than the 15-question pilot comparisons above; the same-dataset pilot comparison shows +0.000 recall change from reranking alone.

9. Agent Evaluation

9.1 Routing and Tool Selection

Final run, all 28 cases in `evaluation/golden_dataset/v2/cases.jsonl`:

Metric	Passed / Total	Rate
Route correct	28/28	100%
Tool selection correct	28/28	100%
Tool arguments correct	28/28	100%
Tool call order correct	28/28	100%
Required-tool recall	50/50	100%

`route_confusion` on this run shows **zero cross-route mistakes** across all 11 distinct expected-route patterns in the dataset — single specialists, 2- and 3-step multi-agent chains, and FINISH.

9.2 Correctness, LLM-as-Judge, and Pricing

Metric	Passed / Total	Rate
Guardrail correct	28/28	100%
Approval correct	10/10	100%
Project state correct	16/16	100%
Cross-checks (e.g. code reuses wiring pins)	2/2	100%
Recovery correct	1/1	100%
Task completion (LLM judge)	14/15	93.3%

LLM as a judge. `evaluation/judge.py` grades whether the final free-text answer actually accomplished the stated goal — the one thing route/tool/ project assertions cannot check, since an answer can use every right tool and still fail the task by describing success without delivering it. It is opt-in per case (15 of 28 declare a goal) and uses structured JSON-schema output rather than the default function-calling extraction, since the underlying reasoning model otherwise sometimes emits chain-of-thought as plain text that the provider cannot parse into a schema.

The judge itself had a real blind spot, found and fixed during this project's evaluation cycle: it originally saw only the query, the goal, and the answer text — not what the turn actually produced. A rendered 3D preview reaches the user through the API's image mechanism, not by the model describing it in prose, so the judge was failing cases that had, in fact, succeeded. It now receives an evidence block (images delivered, saved model/code/wiring artifacts) with an explicit instruction to treat that as real proof of delivery. The one remaining judge failure — `rag_servo_pwm` — is a genuine, narrow factual-precision miss: the answer states a 0.5–2.5ms pulse-width range where the goal expects the ~1–2ms "typical hobby servo" range, left as a real signal rather than loosened.

Pricing. `evaluation/cost_tracking.py` prices every graph run at standard on-demand rates per provider:

Model	Role	\$/M input	\$/M output
<code>openai/gpt-oss-120b</code>	supervisor	0.15	0.60
<code>openai/gpt-oss-safeguard-20b</code>	guardrails, judge	0.075	0.30
<code>qwen/qwen3.6-27b</code>	coding, wiring	0.60	3.00
<code>gemini-3.1-flash-lite</code>	RAG, visualization	0.25	1.50
<code>nvidia/nemotron-3-ultra-550b-a55b</code>	OpenRouter fallback	0.50	2.20

Final 28-case run: **\$0.158 total, \$0.0056/case mean, 10,143 tokens/case mean**. A model missing from the pricing table reports `cost_usd: null` rather than silently costing \$0, so a pricing gap stays visible instead of hidden.

9.3 Held-Out Validation — Is the 100% Real?

A 100% pass rate on a set iterated against all session is a fair thing to distrust — every fix made during this project was verified by re-running the *same* 28 cases. To check whether that reflected genuine correctness or overfitting to those specific phrasings, a second, deliberately harder **41-case held-out set** was built (`evaluation/golden_dataset/v2/cases_holdout.jsonl`), using:

- 17 wiring cases on **components and board combinations never touched during this project's debugging** — an IMU, a motor driver, a stepper motor, a keypad, an RFID reader, a relay, an RGB LED, and others, spread across all four supported boards — with ground truth computed by calling the real pin allocator directly, not hand-predicted.
- Reworded RAG, conversation, and guardrail phrasing on the same underlying topics, plus new adversarial prompt-injection variants.
- Deeper multi-agent chains than the original set exercises, including 3-specialist chains with two sequential human approvals.

Result: 39/41 (95.1%) — genuinely lower than the main set, exactly as a meaningful stress test should produce. Both remaining failures are fully characterized, not mysterious: one MCP transport drop (`anyio.BrokenResourceError` mid-call, which recurred once more on a rerun of the *original* 28-case set too, confirming it is a real, low-frequency MCP reliability gap independent of dataset difficulty — see §11); and one case where the coding agent referenced the wrong pin, caught its own error via the pin-consistency cross-check, and correctly asked for a second approval round to submit corrected code — the test script only scripted one approval turn, so this was a test-authoring gap, not a system bug, and a genuinely reassuring finding that the self-correction path works as designed.

The held-out set also caught two real product bugs the original 28 cases never exercised, both fixed during this evaluation cycle: `wiring_agent` was found writing example firmware code directly into its own answer when a single message asked for wiring *and* code, bypassing `coding_agent` and therefore its human-approval gate entirely (fixed by scoping `WIRING_PROMPT` explicitly to wiring only); and `wiring_agent` intermittently skipped `format_wiring_plan` and hand-wrote a pin table instead of using the tool's validated output (fixed by strengthening the prompt to require the tool call be the source of the final table).

Conclusion: the original 28-case 100% was real for what it tested, but was inflated by having been iterated against directly. The held-out set's ~95%, with fully characterized and mostly non-repeating failures, is the more honest number for this system's actual reliability.

10. Failure Case Analysis

Six cases are documented here — twice the minimum required — because each illustrates a genuinely different failure class, spanning both the RAG side and the agent side of the system.

1. Mislabeled artifact field crashes a natural follow-up — design failure. Asking "can I see the picture?" right after a 3D model was rendered returned an uncaught error. `project["model_artifact"]` was being set to the *rendered preview PNG path* rather than the model's own `.json` path, despite the field name

implying the latter; asked to "show the picture," the specialist's only lever was to re-render using the wrong path, which failed a validation check that wasn't caught before it escaped the tool. Fixed by storing the correct path, hardening the tool's error handling, and adding a supervisor rule so a plain "show me again" is answered directly instead of re-invoking the specialist.

2. A silent empty LLM response erases itself from history — model failure amplified by a design gap.

A rare empty completion from a specialist produced an empty final answer. Two independent parts of the pipeline both silently drop empty content — the persistence layer only saves a truthy answer, and the supervisor's own routing context skips blank messages — so the failed turn vanished from *both* the chat history and the model's own memory of the conversation. The very next message, a reasonable follow-up, looked like a non-sequitur and was refused as out of scope. Fixed by substituting a clear "completed without producing a response" message whenever a specialist returns nothing, so the exchange is always visible to both history and future routing decisions.

3. The evaluation judge is blind to non-text delivery — design failure in the evaluation harness itself.

Covered in full in §9.2: images are delivered independently of what the answer text says, but code is not, and the judge originally couldn't tell the two cases apart. Fixed on both sides — the judge now receives delivery evidence, and the coding agent's prompt now requires the actual code in its answer.

4. An A2A client follows the agent card's advertised URL, not the one it was given — infrastructure/design interaction. Every purchasing-related evaluation case failed when run outside Docker with a DNS error, despite the target container being reachable. System B's A2A server advertises its own address as a Docker-internal hostname (deliberately, so *other containers* can reach it by service name); the A2A client fetches that advertised card first and then routes real calls to whatever it says — not the address it was originally given. Fixed for local, non-Docker evaluation with a `/etc/hosts` entry mapping that hostname back to localhost; the Docker-internal path itself needed no change.

5. A leftover from an in-flight rename crashes every approval case — incomplete refactor. A parameter rename (`payment_credential_id` → `payment_method_id`) was applied consistently across the backend and most of the frontend but missed one call site in the evaluation harness, crashing every scripted approval/rejection case with a `TypeError` and collapsing approval-correctness to zero. A one-parameter fix once traced — notable mainly as a reminder that an evaluation harness is itself part of the system that needs to track a refactor, not an external observer immune to it.

6. RAG: a genuine retrieval/reference mismatch, not a bug. One RAG evaluation case (`FN-US-01`) scored context precision and recall of 0 while faithfulness and relevancy stayed high — the generated answer was well-supported by *something*, but that evidence didn't align with the golden reference's expected span. Documented as needing manual reference/context review rather than a retrieval fix — a **model/data alignment issue**, not a design or prompt failure, since the pipeline behaved correctly given what it actually retrieved.

11. Known Limitations

- **MCP transport reliability.** Occasional `BrokenResourceError` on the streamable-HTTP connection to the MCP server, observed at a low but nonzero rate across evaluation runs. The A2A connection to System B has bounded retries; this connection does not yet.
 - **RAG context precision (0.779 final)** is the weakest RAGAS dimension — a deliberate recall-favoring design tradeoff, not an unexamined gap.
 - **Board-version ambiguity.** The source documentation covers multiple board variants with similar naming; a question that doesn't name the board explicitly can retrieve the wrong variant's pinout. Needs either a clarification turn or a required board field — not yet implemented.
 - **Voice interface is one-directional.** Speech-to-text (faster-whisper) works end-to-end on input; there is no text-to-speech on output.
 - **No fine-tuned classifier** was attempted as a tool.
 - **Multi-item purchases require repeated approvals by design** — buying three different components means three separate proposal/approval cycles, never a bulk auto-purchase. This is intentional (see §6.2), but it does mean a "buy X, Y, and Z" request takes multiple turns to fully resolve rather than one.
-

12. Conclusion and Future Work

This project set out to demonstrate that independent services can cooperate across a real network boundary, that a RAG pipeline's design choices can be justified with measured numbers rather than intuition, that an agent system's routing and tool use can be proven correct rather than merely demonstrated, and that "correct" claims should be stress-tested rather than taken at face value — which is precisely why the held-out validation in §9.3 exists, and why it was worth building even after the original set already scored 100%.

The clearest signal of genuine engineering process in this project is not any single metric — it is the fact that pushing on the metrics with a purpose-built adversarial dataset surfaced two real, fixable bugs the original evaluation had never exercised, and that every one of the six failure cases in §10 has a specific, defensible answer to "was this a model failure, a prompt failure, or a design failure, and why."

Natural next steps: add a retry wrapper to the MCP client to close the one remaining reliability gap; extend board-aware disambiguation to the RAG pipeline; and, time permitting, complete the voice interface with a text-to-speech leg to match the existing speech-to-text path.